

JAVA HASHSET

The Art of Unique Collections

CODEHIVE ACADEMY

Mastering Hash-Based Data Structures

1. What is a HashSet?

A HashSet is a collection that stores **unique elements**. It belongs to the `java.util` package and implements the Set interface. Unlike an ArrayList, a HashSet does not maintain the order of its elements.

Key Characteristics:

- **No Duplicates:** Every element must be unique.
- **Unordered:** It does not guarantee that the order will remain constant over time.
- **Null Support:** It permits one null element.
- **Fast:** Offers constant time performance for basic operations.

BASIC IMPLEMENTATION

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<String> fruits = new HashSet<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Cherry");
        fruits.add("Apple"); // Ignored!

        System.out.println(fruits);
    }
}
```

2. Manipulation & Discovery

The HashSet provides a streamlined set of methods to manage your unique data efficiently.

Method	Logic
<code>add(val)</code>	Adds element if not present. Returns false if duplicate.
<code>contains(val)</code>	Returns true if the item exists (Lightning fast).
<code>remove(val)</code>	Removes specific item.
<code>size()</code>	Returns count of elements.
<code>clear()</code>	Wipes the entire set.

METHOD DEMONSTRATION

```
// Check if a value exists
System.out.println(fruits.contains("Banana")); // true

// Remove a value
fruits.remove("Cherry");

// Size of the set
System.out.println("Total: " + fruits.size());
```

3. The Uniqueness Guarantee

How does HashSet know if "Apple" is already there? It uses a combination of two methods every Java object possesses:

Step 1: hashCode()

Java calculates a "Hash Code" (a numeric ID) for the object. This tells Java which "bucket" the object should go into.

Step 2: equals()

If two objects have the same Hash Code (a collision), Java uses `equals()` to check if they are actually the same object or just coincidental neighbors.

Visual Mapping:

"Apple" → Hash 101 → [Bucket A]
"Banana" → Hash 205 → [Bucket B]
"Apple" → Hash 101 → [Bucket A] (Already occupied!)

4. Under the Hood: The HashMap

Did you know? Every HashSet in Java is actually a hidden HashMap!

When you call `fruits.add("Apple")`, the HashSet does this behind the scenes:

```
// Internal Java Source (Simplified)
public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}
```

It stores your value as a **Key** in a HashMap and uses a dummy object called `PRESENT` as the value. Since HashMap keys must be unique, the HashSet inherits that property automatically.

5. Performance Benchmarks

Why do developers love HashSets? **Speed.**

Search: $O(1)$

Insert: $O(1)$

Delete: $O(1)$

Compare this to an **ArrayList**:

- To find a specific item in an ArrayList, Java might have to check every single element ($O(n)$).
- In a HashSet, Java jumps directly to the bucket where the item belongs. Even with 1 million items, the search time remains nearly identical.

6. Iterating Over a Set

Since HashSets don't have indexes (you can't do `fruits.get(0)`), you must use either a For-Each loop or an Iterator.

ITERATION TECHNIQUES

```
// Method 1: For-Each
for (String i : fruits) {
    System.out.println(i);
}

// Method 2: Lambda (Modern)
fruits.forEach(fruit -> System.out.println(fruit));
```

Warning: Never try to remove elements from a Set while looping using a standard For-Each loop, or you will trigger a `ConcurrentModificationException`. Use an Iterator for safe removal during loops.

7. Alternative Set Implementations

Choosing the right Set depends on your requirements for order and performance.

Implementation	Ordering	Speed
HashSet	None	Fastest
LinkedHashSet	Insertion Order	Moderate
TreeSet	Sorted (Natural)	Slower ($O(\log n)$)

8. Real World Case: Unique Visitors

Imagine you are tracking unique IP addresses visiting your website. You don't care how many times they visited, just how many unique people came.

```
HashSet<String> uniqueVisitors = new HashSet<>();

void onVisit(String ipAddress) {
    uniqueVisitors.add(ipAddress); // Duplicates automatically ignored
}

int getCount() {
    return uniqueVisitors.size();
}
```

Using a List would require a `contains()` check before every addition, making the app slower as the user base grows. HashSet handles this natively and instantly.

9. Summary & Best Practices

1. Use **HashSet** when you need a unique collection and don't care about order.
2. Always ensure the objects you store have robust `hashCode()` and `equals()` implementations.
3. Set the **Initial Capacity** if you know your set will be large, to avoid expensive re-hashing.
4. Understand that HashSets consume more memory than Arrays due to the bucket structure.

The Java Collections Series | Volume 06

© 2026 CodeHive Academy